

Correlation and Data Augmentation

Supplementary notes for Week 1, written as continuous reading rather than slides

These notes sit alongside the Week 1 lecture on *Data Analysis and the ML Workflow*. The lecture gave you the shortest possible version of two ideas: that Pearson and Spearman can disagree violently on the same two columns, and that synthetic training rows must be manufactured after the split and never before it. Both of those statements are true, and both of them hide a great deal.

The purpose of this document is to open them up. Part I asks what a correlation coefficient is actually measuring, why the classical coefficients fail in specific and predictable ways, and what the last fifteen years of statistics have produced to replace them. Part II asks why augmentation works at all, walks through the standard techniques for images, text, tables, audio, time series and graphs, and then takes the modern generative methods seriously, including the ways they can quietly poison a project. Part III is practical: how to run every example here on Google Colab, on Hugging Face, and on a laptop with Ollama, at no cost.

Nothing here is assessed directly. All of it will make the lab, the coursework and Weeks 2 to 11 easier. Read it with a notebook open.

Contents

Contents

Part I · Correlation	1
1 What a correlation coefficient is actually measuring	1
2 Pearson in detail, and what its assumptions actually buy you	1
2.1 The coefficient and the coefficient of determination	1
2.2 Assumptions, and which ones matter for what	1
2.3 Confidence intervals: the Fisher transform	2
2.4 Significance is not strength	2
3 Rank-based correlation: Spearman and Kendall	2
3.1 Spearman's rho	2
3.2 Kendall's tau	3
3.3 Choosing between them	3
4 Correlation when the variables are not both continuous	4
5 Partial and semi-partial correlation	4
6 How correlation lies: the standard failure modes	5
6.1 It cannot see anything that is not a line	5
6.2 Simpson's paradox and the ecological fallacy	6
6.3 Restriction of range and attenuation	6
6.4 Correlation matrices and multiple testing	7
6.5 Time series	7
7 Modern dependence measures	7
7.1 Distance correlation	7
7.2 Chatterjee's xi	8
7.3 Mutual information and its estimators	8
7.4 Kernel methods: HSIC	9
7.5 MIC, and a cautionary tale about new statistics	9
7.6 Practical measures for mixed tables	9
7.7 Multivariate association	9
8 From association to causation	10

9 A working procedure, and code	10
Part II · Data Augmentation	12
10 Why augmentation works at all	12
10.1 The invariance argument	12
10.2 The vicinal risk argument	12
10.3 The regularisation argument	12
11 The two rules that govern every technique	13
12 Images	13
12.1 The classical toolkit	13
12.2 Mixing methods	14
12.3 Learned augmentation policies	14
12.4 Libraries	14
13 Text	15
13.1 Surface-level methods	15
13.2 Back-translation	15
13.3 Contextual and model-based methods	15
13.4 Generating data with large language models	16
13.5 Counterfactual augmentation for fairness	16
14 Tabular data	16
14.1 Why tables are hard	16
14.2 The SMOTE family	17
14.3 The alternative you should try first	18
14.4 Noise-based and generative methods	18
15 Audio, time series and graphs	19
15.1 Audio	19
15.2 Time series	19
15.3 Graphs	20
16 Augmentation as the engine of modern representation learning	20
17 Where augmentation goes wrong	20
17.1 Leakage, which is the one that will cost you marks	20

17.2 Evaluating on synthetic data	21
17.3 Distribution shift and over-augmentation	21
17.4 Bias amplification	21
17.5 Model collapse	22
18 Governance and documentation of augmentation	22
Part III · Playing with this yourself	23
19 Google Colab	23
19.1 What it gives you	23
19.2 The first five things to do in a new notebook	23
19.3 Things that will catch you out	24
20 Hugging Face	24
20.1 What is actually on it	24
20.2 Datasets	24
20.3 Back-translation in ten lines	25
20.4 Contextual word substitution	25
20.5 Model cards, dataset cards and licences	26
21 Ollama, or running a model on your own machine	26
21.1 Why bother when Colab exists	26
21.2 Getting started	26
21.3 Talking to it from Python	27
21.4 What to use it for, and what not to	27
21.5 Combining the three	28
22 Exercises	28
23 References and further reading	29

Part I · Correlation

1 What a correlation coefficient is actually measuring

Start with covariance, because everything else in this part is a variation on it. Given paired observations (x_i, y_i) for $i = 1 \dots n$, the sample covariance is

$$\text{cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

Read the summand as a question asked of every single data point: when this observation was above average on x , was it also above average on y ? If the answer is usually yes, the products are usually positive and the sum grows. If above average on x tends to come with below average on y , the products are negative. If there is no pattern, positives and negatives cancel and the sum drifts towards zero.

Covariance has one fatal presentational problem. Its units are the product of the units of x and y , so a covariance between income in euro and tenure in months is expressed in euro-months, a quantity nobody has any intuition about. Worse, rescaling income from euro to cent multiplies the covariance by one hundred without changing anything real. Pearson's coefficient fixes this by dividing by the two standard deviations, which cancels the units and bounds the result:

$$r = \frac{\text{cov}(x, y)}{s_x s_y} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}, \quad r \in [-1, 1].$$

There is a geometric reading that is worth carrying around, because it explains most of Pearson's behaviour. Centre both columns and treat them as vectors in $\mathbb{R}^n$. Then r is exactly the cosine of the angle between those two vectors. A value of $+1$ means they point in the same direction, -1 means opposite directions, and 0 means they are orthogonal. Once you see r as a cosine, it becomes obvious why it can only detect straight-line structure. A cosine is a linear notion, and it has no vocabulary for curves.

The one-sentence version. Pearson's r is the cosine of the angle between two mean-centred columns, which means it is a measurement of how close the cloud of points lies to a single straight line, and it is blind to every other kind of relationship.

2 Pearson in detail, and what its assumptions actually buy you

2.1 The coefficient and the coefficient of determination

Squaring r gives the proportion of the variance in one variable that a straight-line fit to the other accounts for. An r of 0.5 sounds like half of something, but $r^2 = 0.25$, so three quarters of the variation remains unexplained. This is the single most useful sanity check when somebody reports a correlation in a meeting. Halve your enthusiasm, then halve it again.

2.2 Assumptions, and which ones matter for what

Textbooks list four assumptions for Pearson's r , and students often treat them as a checklist to be ticked before the number is allowed to exist. It is more useful to ask what each assumption is

protecting.

Linearity protects the interpretation of the number. If the true relationship is curved, r does not become invalid in a formal sense, it simply answers a question you did not ask. It reports the quality of the best straight-line approximation, which may be a poor summary of a strong relationship. This assumption matters always.

Absence of influential outliers protects the stability of the number. Because the deviations are squared, a point far from the centre of the cloud contributes enormously. In the Week 1 slides, a handful of extreme incomes pushed r down to 0.14 while the underlying association, visible to the rank methods, was 0.82. The same mechanism runs in the other direction: two clusters that have no internal relationship at all can produce a large r purely because one cluster sits up and to the right of the other. This assumption matters always.

Homoscedasticity, meaning roughly constant spread of y across the range of x , protects the honesty of the confidence interval and the p -value rather than the point estimate. A funnel-shaped cloud can still have a meaningful r , but the standard error attached to it will be wrong.

Bivariate normality is the most misunderstood of the four. It is not needed to compute r , and it is not needed for r to be a sensible descriptive statistic. It is needed for the classical significance test and the classical confidence interval to have the properties they claim. If you only want a descriptive number, you can drop it. If you want to say that r is significantly different from zero with a stated error rate, you cannot.

2.3 Confidence intervals: the Fisher transform

The sampling distribution of r is skewed, and increasingly so as the true correlation approaches ± 1 , because the coefficient is bounded and the distribution gets squeezed against the boundary. Fisher's variance-stabilising transform fixes this:

$$z = \operatorname{arctanh}(r) = \frac{1}{2} \ln \left(\frac{1+r}{1-r} \right), \quad \operatorname{SE}(z) \approx \frac{1}{\sqrt{n-3}}.$$

Build a symmetric interval on the z scale, then transform the endpoints back with $\tanh$. The result is asymmetric on the r scale, which is correct. This matters more than it sounds, because a correlation reported without an interval invites everyone to treat a noisy estimate as a fact. With $n = 30$, an observed r of 0.40 has a ninety-five percent interval running roughly from 0.05 to 0.66, which is the difference between a finding and a shrug.

2.4 Significance is not strength

With $n = 10,000$, a correlation of 0.03 is comfortably significant at conventional thresholds and explains nine hundredths of one percent of the variance. With $n = 12$, a correlation of 0.5 is not significant and may still be the most important signal in the dataset. The p -value answers a question about sample size and sampling noise. It does not answer the question you care about, which is whether the association is large enough to act on. Always report the magnitude, always report an interval, and treat the p -value as the least interesting of the three numbers.

3 Rank-based correlation: Spearman and Kendall

3.1 Spearman's rho

Spearman's coefficient replaces each value with its rank within its own column and then computes Pearson's r on the ranks:

$$\rho = r_{\text{Pearson}}(\text{rank}(x), \text{rank}(y)).$$

The consequences follow immediately from that definition. Because the largest value becomes rank n whether it is ten percent above the next value or ten thousand times above it, extreme values lose their leverage entirely. Because ranks preserve order but not spacing, any monotonic relationship, curved or not, maps to a perfectly linear relationship on ranks and yields ρ near ± 1 . And because ranks are invariant under any increasing transformation, ρ computed on income and ρ computed on $\log(\text{income})$ are identical, which removes an entire category of arbitrary analyst choices.

Spearman is therefore the right default for skewed data, for ordinal variables such as the satisfaction column in the running example, and for any situation where you suspect the relationship is monotone but not straight. Its cost is that it discards magnitude information, so if the relationship really is linear and the data really are clean, Spearman is slightly less efficient than Pearson.

3.2 Kendall's tau

Kendall's coefficient approaches the same problem through pairs rather than ranks. Take every pair of observations (i, j) . The pair is *concordant* if the two variables move in the same direction between them, meaning $x_i < x_j$ and $y_i < y_j$, or both inequalities reversed. It is *discordant* if they move in opposite directions. With C concordant pairs and D discordant pairs out of $\binom{n}{2}$ total,

$$\tau_a = \frac{C - D}{\binom{n}{2}}.$$

The interpretation is unusually clean, and this is Kendall's main selling point. The quantity $(\tau + 1)/2$ is the probability that a randomly chosen pair of observations is concordant. If $\tau = 0.6$, then eighty percent of pairs agree on direction. No other correlation coefficient in common use has a statement that direct.

In practice you almost always want τ_b , which applies a correction for tied values in either column, and which is what `scipy.stats.kendalltau` returns by default. Use τ_c when the two variables have a different number of distinct levels, for example a three-level ordinal against a five-level one.

3.3 Choosing between them

Spearman and Kendall usually agree about the existence and sign of a relationship and disagree about its numeric size, with $|\tau|$ systematically smaller than $|\rho|$ for the same data. Do not compare them across studies as though they were the same scale. Prefer Kendall when the sample is small, when there are many ties, or when you want the probabilistic interpretation. Prefer Spearman when the sample is large, when you want something closer in spirit to Pearson, or when computation time matters, since naive Kendall is quadratic in n while Spearman is dominated by the sort.

A cheap and effective diagnostic. Compute Pearson and Spearman on the same pair of columns and look at the gap. If $|\rho| \gg |r|$ you have skew, outliers or curvature, and the rank view is telling the truth. If $|r| \gg |\rho|$ you very likely have a small number of extreme points manufacturing a straight line out of nothing. If they agree, the relationship is probably close to linear and you can proceed. This single comparison catches most of the damage before it enters a model.

4 Correlation when the variables are not both continuous

Real tables are mixed. The running customers table has a continuous income, a discrete tenure, a nominal plan, an ordinal satisfaction and a binary churn target, and a single call to `df.corr()` will silently drop everything that is not numeric. The following coefficients cover the cases that matter and, usefully, most of them are Pearson's r in disguise.

Pairing	Use	What it is
continuous, continuous	Pearson, Spearman, Kendall	as above
binary, continuous	point-biserial r_{pb}	algebraically identical to Pearson with the binary coded 0/1; equivalent in spirit to a two-sample t -test
binary, binary	phi coefficient ϕ	Pearson on two 0/1 columns; equals $\sqrt{\chi^2/n}$ for a 2×2 table
nominal, nominal	Cramér's V	$\sqrt{\chi^2 / (n \min(k-1, m-1))}$, bounded in $[0, 1]$, symmetric, no direction
nominal, nominal	Theil's U	an asymmetric uncertainty coefficient: the fraction of entropy in y removed by knowing x , which need not equal the reverse
nominal, continuous	correlation ratio η	the share of the variance in the continuous variable explained by group membership, the same quantity as η^2 in one-way ANOVA
ordinal, ordinal	Kendall τ_b , Goodman–Kruskal γ	pair-counting methods built for ties
any, any	mutual information, ϕ_K	see Section 7

Two practical remarks. First, Theil's U being asymmetric is a feature rather than a defect. Knowing a customer's postcode may tell you a great deal about their county while knowing the county tells you almost nothing about the postcode, and a symmetric coefficient is forced to average those two very different facts into one misleading number. Second, when you one-hot encode a nominal variable and then correlate the resulting dummy columns against a target, you are computing a set of point-biserial correlations, one per category. That is a legitimate and often informative thing to do, but it is not the same as a single association measure for the variable as a whole.

5 Partial and semi-partial correlation

A raw correlation between two variables includes every route by which they might move together, including routes that pass through a third variable. Partial correlation removes that third variable from both. Formally, regress x on z and keep the residuals, regress y on z and keep the residuals, then correlate the two residual series. For a single control variable there is a closed form:

$$r_{xy \cdot z} = \frac{r_{xy} - r_{xz} r_{yz}}{\sqrt{1 - r_{xz}^2} \sqrt{1 - r_{yz}^2}}.$$

The semi-partial, or part, correlation removes z from only one of the two variables, which is the quantity implicitly reported when you ask how much a predictor adds to a regression that already contains the others.

Partial correlation is the workhorse behind the observation that ice cream sales and drownings correlate. Control for temperature and the association largely disappears, because temperature

was driving both. It is also the engine inside Gaussian graphical models, where a zero partial correlation between two variables given all the others implies conditional independence under a joint normality assumption, and this is what the graphical lasso estimates on high-dimensional data.

Controlling for the wrong variable is worse than not controlling. If z sits on the causal path from x to y , conditioning on it removes exactly the effect you were trying to measure. If z is a common consequence of x and y , a so-called collider, conditioning on it can *create* a correlation between two genuinely independent variables. Deciding what to control for is a causal question, not a statistical one, and no amount of data will answer it for you. Section 8 returns to this.

6 How correlation lies: the standard failure modes

6.1 It cannot see anything that is not a line

Figure 1 shows six clouds of points. Each panel reports five coefficients on the same data. The panels are ordered so that the classical measures look increasingly foolish.

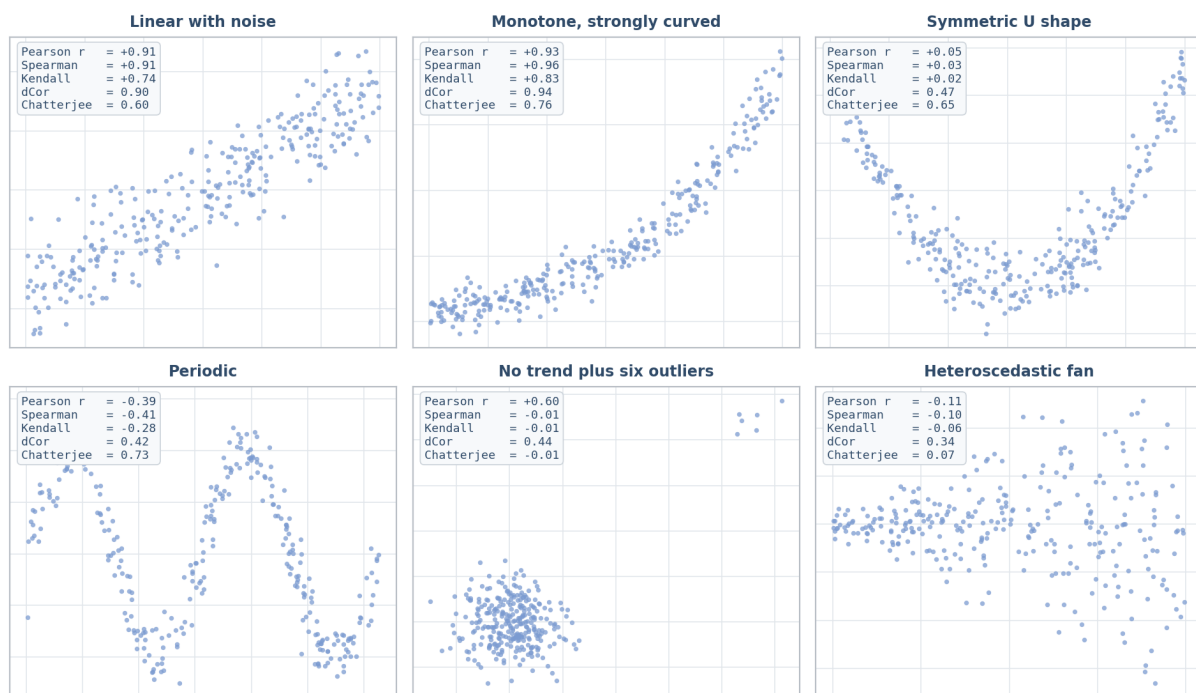


Figure 1: Six relationships, five coefficients each. Distance correlation and Chatterjee's ξ are introduced in Section 7. Note that the U shape and the periodic pattern are strong, nearly deterministic relationships that Pearson, Spearman and Kendall all score near zero or, worse, score with the wrong sign.

The second panel is the ordinary case where the rank methods beat Pearson: a strongly curved but monotone relationship, where Spearman correctly reports near-perfect association while Pearson is dragged down by the curvature. The third and fourth panels are the case where *all three* classical methods fail. A symmetric U shape has as much upward movement as downward movement, so concordant and discordant pairs cancel and every classical coefficient lands near zero, despite y being an almost deterministic function of x . The periodic panel is worse still, returning a moderate negative number that is a pure artefact of where the sampling window happened to start.

The fifth panel is the outlier trap in its purest form, where six points out of three hundred manufacture $r = +0.60$ from a cloud that has no internal relationship at all. Notice that Spearman and Kendall correctly report zero, and notice also that distance correlation reports 0.44. That is worth dwelling on. The modern dependence measures are not robust to outliers by default; they are sensitive to *any* departure from independence, and a tight cluster of six points far from everything else is a genuine departure from independence. Robustness and sensitivity are different properties, and you have to decide which one you want.

The final panel has no trend in the mean at all, only a spread that grows with x . Every classical coefficient reports approximately zero, and every classical coefficient is right that there is no linear or monotone trend. They are also missing something real, because x clearly tells you something about y , specifically about its variance. Distance correlation detects it.

6.2 Simpson's paradox and the ecological fallacy

A correlation computed on pooled data can have the opposite sign to the correlation within every subgroup. This is not a rare curiosity; it appears constantly in customer data, medical data and admissions data.

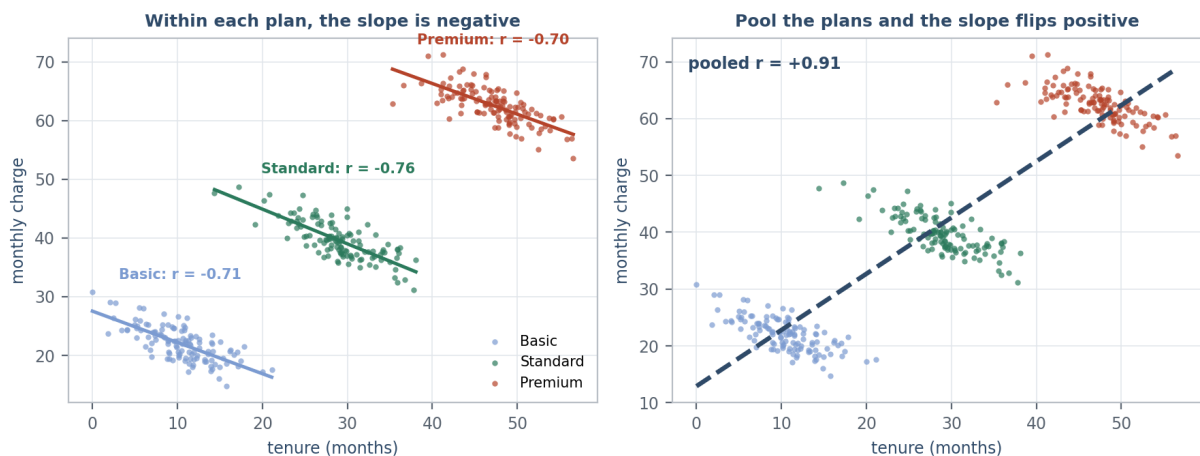


Figure 2: The same six hundred customers. Within each plan, longer tenure comes with a lower monthly charge, because long-standing customers hold legacy pricing. Pooled across plans the sign reverses, because Premium customers have both longer tenure and higher charges. A model fitted to the pooled data learns the wrong sign.

The mechanism is a lurking variable that shifts both the x and the y location of each subgroup. When the between-group shift is larger than the within-group slope, the pooled slope follows the between-group pattern. Nothing in the coefficient warns you. The only defences are to plot the data with the candidate grouping variables coloured in, and to compute the correlation within strata as well as overall.

A close relative is the ecological fallacy, where a correlation observed at an aggregate level is transferred to individuals. Counties with more immigrants may have higher average literacy while immigrants individually have lower literacy than natives, and both statements can hold simultaneously. Aggregate correlations describe aggregates.

6.3 Restriction of range and attenuation

If you compute the correlation between an entrance exam score and degree performance using only the students who were admitted, you have removed the low end of the exam distribution, and the correlation you obtain will be substantially smaller than the correlation in the full applicant pool. This is restriction of range, and it is endemic in any dataset that exists because of

a selection decision, which describes most corporate data. Loan repayment data only contains people who were granted loans.

Attenuation is the mirror problem. If either variable is measured with error, the observed correlation is biased towards zero by a factor involving the reliabilities of the two measurements. A weak correlation between two noisy survey items may reflect a strong underlying relationship between two constructs.

6.4 Correlation matrices and multiple testing

A call to `df.corr()` on a table with p columns computes $p(p-1)/2$ coefficients. With twenty columns that is one hundred and ninety tests, and at a five percent threshold you should expect around nine or ten to look significant when nothing is going on. Scanning a heatmap for the reddest cell and then reporting its p -value is not analysis, it is a slow-motion way of fooling yourself. If you must test many pairs, control the false discovery rate with Benjamini–Hochberg, or better, pre-register the handful of comparisons you actually care about.

6.5 Time series

Two independent random walks will often display a correlation close to ± 1 , because both are wandering and any two wandering series can appear to wander together over a finite window. This is spurious regression, and the standard remedy is to difference the series until they are stationary and then correlate the differences, or to test formally for cointegration if you believe there is a long-run relationship. For a single series, autocorrelation at lag k measures the correlation of the series with itself shifted by k steps and is the basic diagnostic for seasonality and momentum. Standard errors computed under an independence assumption are badly wrong for autocorrelated data, which is why an apparently overwhelming p -value on time series correlations should never be believed at face value.

7 Modern dependence measures

Everything above is at least eighty years old. The last two decades produced a family of measures designed around a stricter goal: rather than quantifying linear or monotone association, they aim to detect *any* form of statistical dependence, and to equal zero if and only if the two variables are genuinely independent. That is a much stronger guarantee than $r = 0$ provides.

7.1 Distance correlation

Distance correlation, introduced by Székely, Rizzo and Bakirov in 2007, is the most broadly useful member of this family. The construction replaces the values themselves with the pairwise distances between observations. Compute the matrix of distances $a_{ij} = |x_i - x_j|$, then doubly centre it by subtracting row means and column means and adding the grand mean, giving A_{ij} . Do the same for y to obtain B_{ij} . The distance covariance is the average of the elementwise product,

$$\mathcal{V}^2(x, y) = \frac{1}{n^2} \sum_{i,j} A_{ij} B_{ij}, \quad \mathcal{R}(x, y) = \frac{\mathcal{V}(x, y)}{\sqrt{\mathcal{V}(x, x) \mathcal{V}(y, y)}} \in [0, 1].$$

The structural resemblance to Pearson is deliberate: a covariance divided by the square root of two variances. The properties that follow are what make it valuable. Distance correlation is zero if and only if x and y are independent, so a value of zero is genuinely informative in a way that $r = 0$ never is. It handles non-linear and non-monotone relationships without being told about

them. It works when x and y have different dimensions, so you can measure the dependence between a scalar target and a whole block of features. It is always non-negative, which means it reports strength but not direction.

The cost is computational: the naive algorithm is $O(n^2)$ in both time and memory, which becomes uncomfortable beyond roughly twenty thousand rows. An $O(n \log n)$ algorithm exists for the univariate case and is what the `dcor` package uses when it can. In practice, subsample.

7.2 Chatterjee's ξ

Chatterjee's coefficient, published in 2021, is the most interesting recent addition because it is both extremely simple and deliberately asymmetric. Sort the observations by x , breaking ties at random. Let r_i be the rank of the y value that now sits in position i . Then

$$\xi_n(x, y) = 1 - \frac{3 \sum_{i=1}^{n-1} |r_{i+1} - r_i|}{n^2 - 1}.$$

The intuition is worth stating plainly. If y is a function of x , then sorting by x makes the y values change smoothly from one row to the next, so consecutive ranks are close together and the sum of absolute differences is small, pushing ξ towards one. If y is independent of x , sorting by x leaves the y ranks in random order, consecutive differences are large on average, and ξ sits near zero.

This gives it two properties no classical coefficient has. It converges to one for *any* deterministic functional relationship, including the U shape and the sine wave that defeat everything else, which is exactly what Figure 1 shows. And it is asymmetric, so $\xi(x, y) \neq \xi(y, x)$ in general, which correctly captures the fact that x may determine y while y leaves x ambiguous. In the U-shape panel, y is a function of x but x is not a function of y , and ξ registers the difference.

The trade-off is statistical power. For simple monotone relationships with moderate noise, ξ needs more data than Spearman to reach the same confidence, and its values for such relationships look unimpressively small. Treat it as a dependence detector rather than a strength meter.

7.3 Mutual information and its estimators

Mutual information is the information-theoretic answer to the same question, measuring how many bits knowing x saves you when encoding y :

$$I(X; Y) = \iint p(x, y) \log \frac{p(x, y)}{p(x)p(y)} dx dy.$$

It is zero exactly when the variables are independent, it is invariant under any invertible transformation of either variable, and it captures every form of dependence. The difficulty is entirely in the estimation, since the densities are unknown. Naive binning of continuous variables produces estimates that depend heavily on the bin width and are biased upwards. The Kraskov–Stögbauer–Grassberger estimator, based on k -nearest-neighbour distances, is the standard non-parametric choice and is what `sklearn.feature_selection.mutual_info_regression` implements. For high-dimensional or representation-learning settings, neural estimators such as MINE and the InfoNCE bound used in contrastive learning turn the estimation into an optimisation problem, at the cost of the usual neural network caveats about variance and training stability.

7.4 Kernel methods: HSIC

The Hilbert–Schmidt Independence Criterion maps both variables into reproducing kernel Hilbert spaces and measures the cross-covariance there. With a characteristic kernel such as the Gaussian RBF, HSIC is zero if and only if the variables are independent, giving the same guarantee as distance correlation, and in fact the two are closely related. Its practical advantage is that it extends naturally to structured objects, since you only need a kernel: strings, graphs and sets all work. It appears inside independence-regularised training objectives and in some fairness constraints, where you penalise the dependence between a model’s predictions and a protected attribute.

7.5 MIC, and a cautionary tale about new statistics

The Maximal Information Coefficient was published in *Science* in 2011 with the appealing claim of equitability, meaning that it would assign similar scores to equally noisy relationships regardless of their functional form. It attracted immediate and substantial criticism, most prominently from Simon and Tibshirani, who showed that MIC has notably lower statistical power than distance correlation against most alternatives, so that it fails to detect real relationships that simpler methods find. The underlying notion of equitability was also shown to be less well defined than it first appeared. MIC is included here not because you should use it but because the episode is a useful lesson: a new coefficient with a compelling story and a high-profile publication is not automatically an improvement, and power against realistic alternatives is the property that actually matters.

7.6 Practical measures for mixed tables

Two tools deserve mention because they solve the everyday problem of a table with mixed column types. The ϕ_K coefficient, from the `phiK` package, works uniformly across interval, ordinal and categorical variables, captures non-linear dependence, reverts to Pearson for bivariate normal data, and comes with a significance evaluation. The Predictive Power Score takes a frankly pragmatic approach: train a small decision tree to predict y from x alone, cross-validate it, and normalise the score against a naive baseline. It is asymmetric, it handles any column type, and it has no distributional assumptions. It is also not a statistic in the classical sense, its value depends on the model and the cross-validation scheme, and it should be used for screening rather than for anything you intend to report as a finding.

7.7 Multivariate association

When both sides are blocks of variables rather than single columns, canonical correlation analysis finds the pair of linear combinations, one from each block, with maximum correlation, and then repeats orthogonally. Deep CCA replaces the linear combinations with neural networks and is used for cross-modal representation learning. Distance correlation and HSIC both handle the multivariate case directly without needing this machinery.

Measure	Non-linear	Direction	Zero iff indep.	Reach for it when
Pearson r	no	yes	no	the relationship is plausibly linear and clean
Spearman ρ	monotone only	yes	no	skew, outliers, ordinal data
Kendall τ_b	monotone only	yes	no	small n , many ties, want a probability
Distance corr.	yes	no	yes	screening for any dependence, mixed dimensions
Chatterjee ξ	yes	asymmetric	yes	suspect a functional but non-monotone relationship
Mutual info.	yes	no	yes	want bits, or a principled feature filter
HSIC	yes	no	yes	structured data, or a differentiable penalty
ϕ_K	yes	no	approx.	a mixed-type table and you want one matrix

8 From association to causation

The slogan that correlation does not imply causation is repeated so often that it stops carrying information. The more useful formulation is that an observed association between x and y is consistent with at least five distinct explanations, and the data alone cannot distinguish between them: x causes y ; y causes x ; some third variable z causes both; the sample was selected in a way that induces the association, which is collider bias; or it is chance, especially after many comparisons.

Distinguishing between these requires assumptions that come from outside the dataset. The modern framework for stating those assumptions precisely is the directed acyclic graph, where you draw your beliefs about which variables cause which, and then read off which set of variables must be conditioned on to obtain an unbiased estimate. The backdoor criterion tells you when a set is sufficient. Once you have a valid adjustment set, the practical estimation can proceed by regression, by propensity score methods, or by doubly robust estimators. Where no valid adjustment set exists, instrumental variables, regression discontinuity and difference-in-differences exploit specific structures to recover a causal effect. For time series, Granger causality tests whether past values of x improve the prediction of y beyond y 's own past, which is a statement about predictive precedence rather than causation despite the name; transfer entropy is its information-theoretic generalisation.

None of this is Week 1 material, and it is not assessed here. It is worth knowing that it exists, because the honest answer to "is this correlation causal?" is almost never "let me compute a different coefficient".

9 A working procedure, and code

When you meet a new pair of columns, the following sequence costs about two minutes and prevents most of the errors in this part.

1. Plot the scatter first. Every coefficient is a compression of the picture, and you should see the picture before you accept the compression.
2. Compute Pearson and Spearman together and compare them. A large gap is a diagnostic, not a nuisance.
3. If both are near zero, compute distance correlation or Chatterjee's ξ before concluding that there is no relationship.

4. Report an interval, not just a point estimate.
5. Colour the scatter by your main categorical variables to check for Simpson's paradox.
6. Ask what a third variable might be doing, and whether the sample exists because of a selection process.

Listing 1: *A reusable comparison of classical and modern coefficients*

```
import numpy as np, pandas as pd
from scipy import stats
import dcor # pip install dcor

def chatterjee_xi(x, y, rng=np.random.default_rng(0)):
    """Chatterjee (2021). Asymmetric: how nearly is y a function of x?"""
    x, y = np.asarray(x, float), np.asarray(y, float)
    n = len(x)
    order = np.argsort(x + rng.random(n) * 1e-9) # random tie-breaking
    r = stats.rankdata(y[order], method="max")
    return 1.0 - 3.0 * np.abs(np.diff(r)).sum() / (n*2 - 1)

def association_report(x, y):
    x, y = np.asarray(x, float), np.asarray(y, float)
    ok = np.isfinite(x) & np.isfinite(y) # pairwise deletion, stated openly
    x, y = x[ok], y[ok]
    r, p_r = stats.pearsonr(x, y)
    rho, p_s = stats.spearmanr(x, y)
    tau, p_t = stats.kendalltau(x, y)
    z = np.arctanh(r); se = 1 / np.sqrt(len(x) - 3)
    lo, hi = np.tanh(z - 1.96 * se), np.tanh(z + 1.96 * se)
    return pd.Series({
        "n": len(x),
        "pearson_r": round(r, 3),
        "pearson_lo": round(lo, 3),
        "pearson_hi": round(hi, 3),
        "spearman_rho": round(rho, 3),
        "kendall_tau": round(tau, 3),
        "distance_cor": round(dcor.distance_correlation(x, y), 3),
        "xi_x_to_y": round(chatterjee_xi(x, y), 3),
        "xi_y_to_x": round(chatterjee_xi(y, x), 3),
    })

# the gap between pearson_r and spearman_rho is the first thing to read
print(association_report(df["income"], df["monthly_charge"]))
```

Missing values deserve a decision. `pandas.corr()` defaults to pairwise deletion, which means different cells of the same correlation matrix are computed on different subsets of rows. The resulting matrix can fail to be positive semi-definite, which breaks anything downstream that expects a valid covariance structure, such as PCA or a Gaussian graphical model. Decide explicitly between listwise deletion, pairwise deletion and imputation, and write the decision into your datasheet.

Part II · Data Augmentation

10 Why augmentation works at all

The lecture presented augmentation as a way of manufacturing more training rows when you do not have enough. That is the effect, but it is not the mechanism, and confusing the two leads directly to bad augmentation choices. Copying a row twice also gives you more rows and achieves nothing. Something else has to be happening.

10.1 The invariance argument

A model learns a function from inputs to outputs. In almost every real problem you know things about that function before you see any data. You know that a photograph of a cat flipped left to right is still a photograph of a cat. You know that a sentence with one word replaced by a synonym still carries the same sentiment. You know that a customer whose income is recorded as fifty thousand and one euro rather than fifty thousand euro has the same probability of churning. Each of these is a statement that the target is *invariant* to some transformation of the input.

That knowledge has to get into the model somehow. Occasionally you can build it into the architecture, as convolutional layers build in approximate translation invariance and graph networks build in permutation invariance. Usually you cannot, and augmentation is the practical alternative: rather than constraining the function class, you show the model both the original and the transformed input with the same label, and let it learn the invariance from examples. Seen this way, an augmentation policy is a precise statement of what you believe about the problem. Choosing to include vertical flips in a medical imaging pipeline is asserting that anatomy has no up and down, which is false, and the model will duly learn something false.

10.2 The vicinal risk argument

The formal version comes from vicinal risk minimisation. Standard empirical risk minimisation treats your training set as a collection of point masses and minimises the average loss over exactly those points. Vicinal risk minimisation replaces each point mass with a small distribution in its neighbourhood, a vicinity, and minimises the expected loss over those neighbourhoods instead. Augmentation is precisely a way of drawing samples from those vicinities. The choice of transformation is the choice of what neighbourhood you believe surrounds each training example. Mixup, discussed below, takes this literally and defines the vicinity as the line segments between pairs of training examples.

10.3 The regularisation argument

Empirically, augmentation acts as a regulariser. It makes memorisation harder, because the model never sees exactly the same input twice, so it is pushed towards features that survive the transformations. It flattens the loss landscape and typically improves calibration. It usually reduces variance at some cost in bias, which is the classic regularisation trade. The corollary is that augmentation helps most when your model is over-parameterised relative to your data, and can actively hurt when you already have abundant data and a small model, where all it does is make the optimisation problem harder.

Three things augmentation is not. It is not a way of creating information that was not in the data. It is not a substitute for collecting more or better data, which remains the highest-value intervention in almost every project. And it is not a fix for a badly framed problem or a badly chosen target.

11 The two rules that govern every technique

Rule one: the transformation must preserve the label. The lecture's example is the cleanest one. Flip a photograph of a handwritten six horizontally and it becomes something close to a nine, so horizontal flipping on MNIST-like data is not augmentation, it is label noise deliberately injected into your training set. The same failure hides in less obvious places. Rotating a chest X-ray by ninety degrees destroys the anatomical prior that the radiologist relies on. Replacing "not good" with "no good" is fine while replacing "not" with "very" inverts the sentiment. Adding Gaussian noise to a tabular age column is harmless at a standard deviation of one year and produces negative ages at a standard deviation of fifty.

Rule two: the transformation must produce inputs that could plausibly occur at deployment. An augmentation that satisfies rule one but generates inputs the model will never meet in production spends capacity on a distribution that does not matter, and in the worst case shifts the model away from the real one. If your cameras are always mounted upright, training on upside-down images is wasted effort. If your billing system enforces two decimal places, synthetic rows with six decimal places are teaching the model about a world that does not exist.

Everything else in Part II is an application of these two rules to a particular data type.

12 Images

Images are where augmentation was developed and where it is most effective, because the invariances are numerous, obvious and easy to implement.

12.1 The classical toolkit

Geometric transformations change where things are: horizontal flip, vertical flip, rotation, random resized crop, translation, scaling, shearing and general affine or perspective warps. Random resized crop is the single most valuable of these for natural images, since it forces the model to recognise objects at varying scales and positions, and it is the backbone of most modern training recipes.

Photometric transformations change how things look: brightness, contrast, saturation and hue jitter, gamma correction, grayscale conversion, solarisation and posterisation. These encode the belief that object identity does not depend on illumination or camera response. They are the natural choice when your deployment conditions vary in lighting, which is almost always.

Noise and degradation adds Gaussian or shot noise, motion blur, defocus blur, JPEG compression artefacts and downsampling. This is where you encode the quality of the sensor you will actually have. A model trained only on clean images and deployed on a cheap webcam will fail in ways that look mysterious until you realise the training distribution never contained a blurry frame.

Occlusion methods remove information rather than transforming it. Cutout and Random Erasing mask out a rectangular region, forcing the model to use the whole object rather than latching onto one discriminative part. This is one of the cheapest ways to improve robustness and it costs a single line.

12.2 Mixing methods

Mixup, introduced by Zhang and colleagues in 2018, takes two training examples and blends both the inputs and the labels linearly:

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha).$$

The resulting image is a ghostly superposition that looks like nothing you would ever photograph, which initially seems to violate rule two, and yet it works reliably. The explanation is that mixup encourages the model to behave linearly between training examples, which suppresses the wildly confident predictions that neural networks otherwise make in the regions between classes. The practical benefits are better calibration, improved robustness to label noise and to adversarial examples, and a consistent small accuracy gain.

CutMix replaces a rectangular patch of one image with a patch from another and mixes the labels in proportion to the patch area, combining the benefits of Cutout and mixup while keeping the local statistics of real images. Manifold Mixup applies the same interpolation to intermediate hidden representations instead of raw pixels. AugMix composes several augmentation chains and blends them, with a consistency loss across the versions, and was designed specifically for robustness to distribution shift.

12.3 Learned augmentation policies

Hand-tuning which transformations to apply, in what order and at what strength is a tedious hyperparameter search. AutoAugment, from Google in 2018, formulated it as a reinforcement learning search over policies and found policies that improved ImageNet accuracy, at a cost of thousands of GPU hours. That cost provoked a sequence of cheaper alternatives. Fast AutoAugment used density matching to cut the search dramatically. Population Based Augmentation learned a schedule rather than a fixed policy. RandAugment made the decisive simplification: discard the search entirely, sample transformations uniformly from a fixed list, and expose only two hyperparameters, the number of transformations applied per image and a single global magnitude. It matched or beat the searched policies. TrivialAugment went one step further by removing the magnitude parameter as well, sampling it uniformly at random, and remains competitive.

The trajectory is instructive and generalises well beyond augmentation. An expensive learned solution was progressively simplified until almost nothing was left, and the almost-nothing worked just as well. Start with RandAugment or TrivialAugment. Only consider a search if you have a genuinely unusual domain and compute to spare.

12.4 Libraries

`torchvision.transforms.v2` is the default for PyTorch users and now handles bounding boxes, segmentation masks and keypoints correctly alongside the image. `Albumentations` is faster, has a wider catalogue including many medical and satellite-specific transforms, and has excellent support for keeping annotations consistent. `Kornia` implements transformations as differentiable GPU operations, which matters if you want augmentation inside the computation graph. For TensorFlow, `KerasCV` covers the same ground.

Listing 2: *A reasonable default image pipeline with Albumentations*

```
import albumentations as A

train_tf = A.Compose([
```

```

A.RandomResizedCrop(size=(224, 224), scale=(0.65, 1.0)),
A.HorizontalFlip(p=0.5), # ask first: is left/right meaningful?
A.Affine(rotate=(-12, 12), shear=(-6, 6), p=0.5),
A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05, p=0.7),
A.OneOf([A.GaussNoise(), A.MotionBlur(), A.ImageCompression(quality_range=(55, 95))], p
        =0.3),
A.CoarseDropout(num_holes_range=(1, 4), p=0.3), # Cutout
A.Normalize(),
])

# Validation and test get resize plus normalise ONLY. No randomness, ever.
eval_tf = A.Compose([A.Resize(256, 256), A.CenterCrop(224, 224), A.Normalize()])

```

13 Text

Text is harder than images because the data are discrete and because tiny edits can invert meaning. There is no text equivalent of a two-degree rotation that is guaranteed to be harmless.

13.1 Surface-level methods

Easy Data Augmentation, published by Wei and Zou in 2019, is the standard baseline and consists of four operations applied to a small fraction of tokens: synonym replacement using WordNet, random insertion of a synonym of a random word, random swap of two words, and random deletion. It is trivial to implement and gives measurable gains on small datasets, typically vanishing once you have a few thousand labelled examples per class. Character-level noise, simulating typos and keyboard-adjacency errors, is valuable when the deployment text is user-generated.

The weakness of all of these is that WordNet synonyms are context-free. Replacing "bank" with "riverbank" in a financial sentence produces nonsense, and replacing an intensifier can change a sentiment label outright.

13.2 Back-translation

Translate the sentence into another language and then back into the original. The round trip preserves meaning, since that is what translation systems optimise, while substantially varying the surface form. Pivoting through a linguistically distant language produces more variation than pivoting through a close one. Chaining two pivots increases diversity further at increasing risk of drift. This remains one of the strongest text augmentation methods, and with Hugging Face's Helsinki-NLP models it is about ten lines of code. Its costs are compute, since you are running two sequence models per example, and a tendency to normalise informal text into something blander than the original.

13.3 Contextual and model-based methods

Masked language model substitution replaces a token with [MASK] and lets BERT or a similar model propose a replacement that fits the context. This solves the synonym problem, because the substitution is conditioned on the surrounding sentence. Guard against it proposing a replacement that flips your label, which is a real risk for sentiment tasks; conditioning the language model on the label, as in conditional BERT augmentation, helps.

Unsupervised Data Augmentation introduced a TF-IDF-aware variant where words with low TF-IDF scores are preferentially replaced, on the reasoning that low-information words carry less

of the label. Paraphrase generation with a sequence-to-sequence model such as T5 or PEGASUS gives whole-sentence rewrites with more variation than token-level edits.

13.4 Generating data with large language models

The current frontier for text is to ask a capable language model to produce the training data outright. Self-Instruct, and the Alpaca dataset that followed it, bootstrapped instruction-following data from a small seed set by prompting a strong model to generate new instructions and responses. Evol-Instruct systematically rewrites prompts to make them harder along several axes, producing a curriculum of increasing difficulty. For classification, you can prompt for examples of a given class, for hard negatives near a decision boundary, or for paraphrases of your existing scarce examples.

This is genuinely powerful for low-resource problems and it comes with four specific hazards. The generated data inherit the biases and the characteristic style of the generating model. Diversity collapses without deliberate effort, since repeated sampling from the same prompt yields near-duplicates, so vary the prompts, raise the temperature and deduplicate aggressively with embedding similarity. The licence terms of commercial model providers frequently restrict using outputs to train competing models, which is a legal question rather than a technical one and which you should check before building a project on it. And unverified generated labels can be confidently wrong, so treat generated data as noisy and either filter it with a verifier or accept that you are training on approximations. Running the generator locally through Ollama, as described in Part III, removes the licensing concern for open-weight models and removes the privacy concern entirely.

13.5 Counterfactual augmentation for fairness

A specific and ethically important use of text augmentation is counterfactual data augmentation: take each training sentence and produce a version with demographic terms swapped, so that "he is a nurse" is accompanied by "she is a nurse" with the same label. This directly attacks the stereotype associations the model would otherwise absorb from the corpus. It connects Week 1 to the fairness material in Weeks 3 and 4, and it is one of the few interventions that addresses bias at the data level rather than patching the model afterwards.

14 Tabular data

Tabular data is where augmentation is least effective and most often misapplied, which makes it the most important section for your coursework, since your project dataset is probably a table.

14.1 Why tables are hard

Images have obvious invariances. Tables have almost none. There is no transformation of a customer record that you know, a priori, leaves the churn probability unchanged, because the columns are heterogeneous, often causally entangled, and constrained in ways that are not written down anywhere. Adding noise to income changes the debt-to-income ratio in the next column. Interpolating between two customers can produce someone aged forty-two with a tenure of sixty months who joined last year. The relationships between columns are exactly what the model is trying to learn, and careless augmentation destroys them.

14.2 The SMOTE family

SMOTE, from Chawla and colleagues in 2002, remains the reference method for class imbalance. For each minority point x_i , find its k nearest minority neighbours, pick one at random, and place a new point at a random position on the segment between them:

$$x_{\text{new}} = x_i + \lambda (x_{z_i} - x_i), \quad \lambda \sim \text{Uniform}(0, 1).$$

The variants exist to fix specific failures of the original.

Variant	What it changes and why
Borderline-SMOTE	synthesises only from minority points that have majority neighbours, concentrating effort near the decision boundary where it matters
SVM-SMOTE	uses an SVM to identify the boundary region and generates there
ADASYN	generates more points for minority examples that are harder to learn, measured by the majority fraction among their neighbours
KMeans-SMOTE	clusters first, then oversamples within clusters, which avoids generating points in empty regions between clusters
SMOTE-NC	handles mixed numeric and categorical columns by interpolating the numeric parts and taking a majority vote for the categorical ones
SMOTEN	for datasets that are entirely categorical, using a value-difference metric instead of Euclidean distance
SMOTETomek, SMOTEENN	oversample, then clean up by removing pairs or neighbourhoods that straddle the boundary

Figure 3 shows what these actually do to a two-dimensional dataset, and it exposes SMOTE's central weakness better than any description. A handful of minority points sit deep inside the majority cloud, most likely mislabelled or simply unusual. SMOTE happily interpolates between them and the main minority cluster, drawing long synthetic bridges straight through majority territory. Every point on those bridges is a confidently labelled minority example in a region that is overwhelmingly majority, and the classifier will try to carve out a path to accommodate them. Borderline-SMOTE largely avoids this by refusing to synthesise from interior points. ADASYN, which deliberately emphasises hard examples, makes it worse.

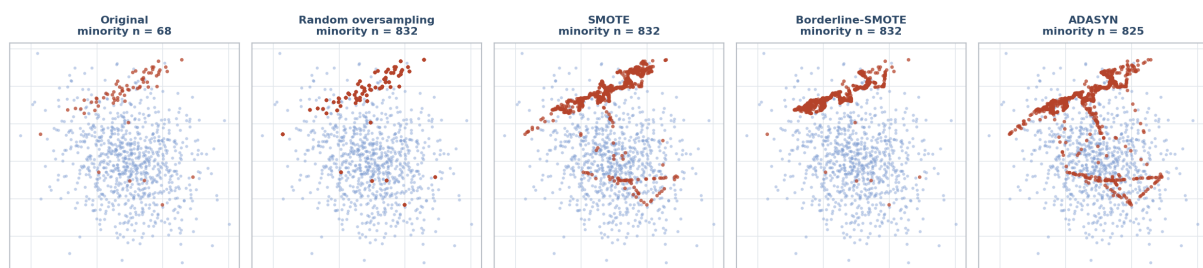


Figure 3: Five treatments of the same imbalanced dataset. Note the long synthetic bridges that SMOTE and ADASYN draw through the majority region, each one an invented minority example in a place where the minority class does not live.

Three further cautions. SMOTE degrades badly in high dimensions, because nearest neighbours become meaningless as dimensionality grows and the interpolations land nowhere useful. It assumes the space between two minority points is itself minority territory, which is false whenever the minority class is multi-modal. And it cannot invent information: every synthetic point is a convex combination of points you already had, so the convex hull of the minority class never grows.

14.3 The alternative you should try first

Before reaching for any resampler, try class weighting. Almost every scikit-learn classifier accepts `class_weight="balanced"`, and gradient boosting libraries accept `scale_pos_weight`. This reweights the loss function so that minority errors cost more, which addresses the same problem without inventing a single data point, without any leakage risk, and without distorting the feature distribution. In a large fraction of published comparisons, class weighting matches or beats SMOTE.

It is also worth questioning whether the imbalance is a problem at all. Imbalance harms accuracy as a metric far more than it harms the model. If you switch to precision, recall, F1, or the area under the precision-recall curve, and if you tune the decision threshold rather than accepting the default of 0.5, many apparent imbalance problems dissolve. Resample because your model genuinely fails to learn the minority class, not because the class counts look uneven in a value counts printout.

14.4 Noise-based and generative methods

Gaussian jitter adds small noise to numeric columns, scaled to each column's standard deviation, and is the tabular analogue of image noise. Swap noise, used to train denoising autoencoders on tabular data, replaces a random subset of cells with values drawn from the same column elsewhere in the dataset, which preserves each marginal distribution while destroying the joint structure the model must then reconstruct.

The generative approach learns the joint distribution and samples from it. CTGAN and TVAE, both from the Synthetic Data Vault project, were designed specifically for tabular data and handle the two things that break naive generative models on tables: multi-modal continuous columns, addressed with mode-specific normalisation, and severely imbalanced categorical columns, addressed with conditional sampling per category. CopulaGAN combines a copula transformation with the GAN. More recently, TabDDPM applies diffusion models to tables with strong reported results, and GReaT fine-tunes a language model on rows serialised as text, which handles mixed types naturally and can condition on any subset of columns.

These are attractive and they carry a specific risk that makes them harder to use than they look. A generative model trained on your data can memorise rows, which means synthetic data is not automatically privacy-preserving, and releasing it under the belief that it is can be a data protection failure. If privacy is the motivation, you need a generator trained with differential privacy guarantees and you need to run membership inference evaluations. The Synthetic Data Vault ships quality and privacy diagnostics for exactly this reason. Use them.

Listing 3: *Tabular augmentation done correctly, inside an imblearn Pipeline*

```
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline as SkPipeline
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline # NOT sklearn's Pipeline

num = ["age", "income", "tenure", "monthly_charge"]
cat = ["plan", "region", "satisfaction"]

prep = ColumnTransformer([
    ("num", SkPipeline([("impute", SimpleImputer(strategy="median")),
                        ("scale", StandardScaler())]), num),
```

```

    ("cat", OneHotEncoder(handle_unknown="ignore"), cat),
])

# SMOTE sits INSIDE the pipeline, so it is refit on each training fold
# and never touches a validation fold.
model = ImbPipeline([
    ("prep", prep),
    ("smote", SMOTE(random_state=0, k_neighbors=5)),
    ("clf", RandomForestClassifier(n_estimators=300, random_state=0)),
])

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          stratify=y, random_state=0)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
print(cross_val_score(model, X_tr, y_tr, cv=cv, scoring="average_precision").mean())
model.fit(X_tr, y_tr) # the held-out test set is still untouched and imbalanced

```

scikit-learn's Pipeline cannot hold a resampler. It has no concept of a step that changes the number of rows, and it applies every step at prediction time, which would resample your test data. You must use `imblearn.pipeline.Pipeline`, which knows to apply resampling steps during fit only and to skip them during predict. This is the single most common technical error in student coursework on imbalanced data.

15 Audio, time series and graphs

15.1 Audio

The classical audio transformations operate on the waveform: time shifting, time stretching without changing pitch, pitch shifting without changing duration, adding background noise at a controlled signal-to-noise ratio, and convolving with a room impulse response to simulate reverberation. Speed perturbation, which resamples at ninety, one hundred and one hundred and ten percent, is a standard and surprisingly effective trick in speech recognition.

SpecAugment, from Google in 2019, changed the field by operating on the spectrogram instead of the waveform. It applies time warping, masks a block of consecutive frequency channels, and masks a block of consecutive time steps. Because it works on the input representation the model actually consumes, it costs almost nothing, and it produced large gains in speech recognition. It is the closest thing audio has to Cutout, and the analogy is exact.

15.2 Time series

Time series augmentation must respect temporal order, which rules out any operation that shuffles points. The standard operations are jittering with additive noise, scaling by a random constant, magnitude warping with a smooth random curve, time warping that stretches and compresses different segments, window slicing that crops a random contiguous subsequence, and permutation of whole segments where segment order is genuinely exchangeable. More elaborate methods generate new series as dynamic time warping barycentric averages of existing ones, and TimeGAN learns to generate sequences that preserve temporal dynamics.

The specific hazard in time series is that augmentation interacts with the split. Time series must be split chronologically, never randomly, and any augmented window that spans the boundary between train and validation is leakage. Generate windows first, assign them to splits by their time index, and only then augment.

15.3 Graphs

Graph augmentation, used heavily in graph contrastive learning frameworks such as GraphCL, consists of node dropping, edge perturbation where edges are added or removed, attribute masking on node features, and subgraph sampling by random walk. The choice of which to use is strongly dataset-dependent, which is a recurring theme: molecules are extremely sensitive to edge perturbation, since removing a bond produces a different molecule, while social networks tolerate it easily.

16 Augmentation as the engine of modern representation learning

It is worth noticing that augmentation has moved from a preprocessing trick to the central mechanism of several major methods.

Self-supervised contrastive learning, as in SimCLR, MoCo, BYOL and DINO, has no labels at all. It creates two augmented views of the same image and trains the network to produce similar representations for them while distinguishing them from other images. The entire learning signal comes from the augmentation policy, which means the policy defines what the representation learns to ignore. SimCLR's ablations showed that the combination of random cropping and colour distortion was essential, and that removing colour distortion let the network cheat by matching colour histograms. If you take one thing from this paragraph, take that: the augmentations *are* the supervision.

Consistency regularisation in semi-supervised learning applies the same idea to unlabelled data. FixMatch produces a weakly augmented version of an unlabelled image, takes the model's confident prediction as a pseudo-label, and trains the model to produce that same label for a strongly augmented version. Unsupervised Data Augmentation does the equivalent for text using back-translation as the strong augmentation. These methods extract a great deal from small labelled sets and are worth knowing about when labelling is your bottleneck.

Test-time augmentation turns the idea around at inference. Run several augmented versions of each test input through the trained model and average the predictions. It reliably improves accuracy and calibration at the cost of a proportional increase in inference time, and it is nearly free to implement. Note carefully that this is not a violation of the rule against augmenting the test set, because you are not training on the augmented data and not changing the labels; you are ensembling predictions over transformations of a single input.

17 Where augmentation goes wrong

17.1 Leakage, which is the one that will cost you marks

Figure 4 makes the point with data that contains no signal whatsoever: five hundred rows, twenty-five columns of pure Gaussian noise, and labels assigned by a coin flip weighted to twelve percent positive. No model can do better than chance on this data, and any procedure that claims otherwise is broken.

The mechanism is straightforward once you see it. Each synthetic minority point is a convex combination of two real minority points. If SMOTE runs before the split, a synthetic point can land in the validation fold while both of its parents sit in the training fold. The model has effectively seen that validation point already, because it is a weighted average of two rows it memorised. With an oversampling ratio large enough to balance a twelve percent minority class, the validation folds fill up with these near-duplicates and the score approaches one.

The same argument applies to every other form of augmentation. Augmenting images before

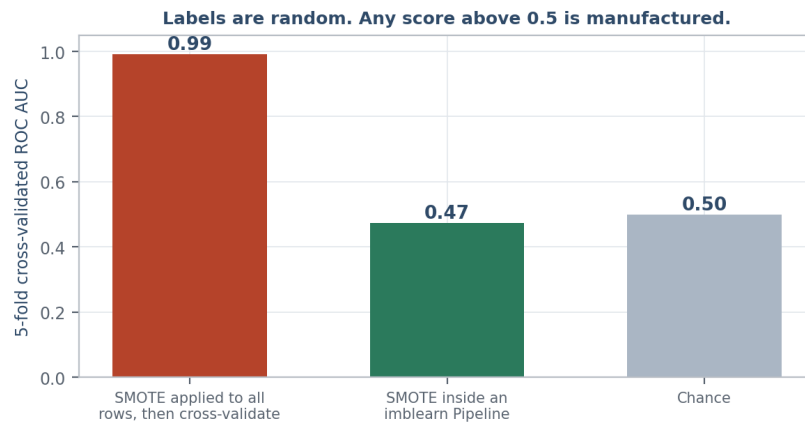


Figure 4: Applying SMOTE to the full dataset and then cross-validating reports an ROC AUC of 0.99 on data where the labels are random. The same model with SMOTE inside an imblearn Pipeline reports 0.47, which is chance within sampling noise. The difference is one line of code.

splitting means rotated copies of a training photograph appear in your test set. Back-translating sentences before splitting means paraphrases of training sentences appear in your test set. Fitting a CTGAN on the full table before splitting means the generator has seen the test rows. The rule is uniform and admits no exceptions: **split first, then augment the training portion only**.

17.2 Evaluating on synthetic data

Related but distinct. Even if you split correctly, it is tempting to report performance on a balanced test set because the numbers look better. Do not. The test set must reflect the deployment distribution, including its imbalance, because that is the only way the numbers mean anything operationally. A model with ninety-five percent accuracy on a balanced synthetic test set may have a precision of four percent when the real base rate is one in a thousand, and precision of four percent is what the business will experience.

17.3 Distribution shift and over-augmentation

Augmentation moves the training distribution away from the real one. A small move is a useful regularisation and a large move is a liability. If your augmented images are so heavily distorted that a human struggles with them, you are training on a different problem. The practical test is to sample twenty augmented examples and look at them, which is a step almost everyone skips. Similarly, if the minority class is two percent of your data, fully balancing it with SMOTE means ninety-eight percent of your minority training examples are synthetic, and the model is learning the interpolation scheme rather than the class. Partial resampling, to perhaps a one-to-three ratio rather than one-to-one, is often better, and the sampling ratio is a hyperparameter to be tuned like any other.

17.4 Bias amplification

If a demographic group is underrepresented, SMOTE and generative models will reproduce that underrepresentation, and by adding volume they can make the resulting bias look statistically better supported than it is. Augmentation multiplies whatever is in your data, including the parts you would rather not multiply. Check subgroup representation before and after augmenting.

17.5 Model collapse

A recent and important finding, published by Shumailov and colleagues in 2024, is that generative models trained recursively on their own outputs degrade, losing the tails of the original distribution first and converging towards a narrow, low-variance approximation of it. For your purposes the lesson is bounded but real: synthetic data should supplement real data rather than replace it, generation chains should not be iterated, and the proportion of synthetic data in any training set is a quantity worth tracking.

18 Governance and documentation of augmentation

This connects directly to LO5 and to the datasheet you are asked to produce in the lab. Augmentation is a transformation of the evidence base on which a model is built, and an undocumented transformation is an unauditable one. Add the following to your datasheet.

1. **What fraction of the training data is synthetic**, per class and, where relevant, per demographic subgroup. This single number is the one an auditor will ask for first.
2. **Which transformations were applied**, with their parameters and the random seed, so that the augmented set can be regenerated exactly.
3. **The justification for label preservation**. Write one sentence per transformation explaining why you believe it leaves the label intact. If you cannot write the sentence, remove the transformation.
4. **Where in the pipeline augmentation occurs**, with explicit confirmation that it runs after the split and touches training data only.
5. **For generative augmentation, the provenance of the generator**: which model, which weights, which licence, what it was trained on, and what privacy evaluation was performed.
6. **Evaluation on real data**, stated plainly, with the class balance of the test set reported.

The regulatory direction of travel makes this less optional than it once was. The EU AI Act imposes data governance requirements on high-risk systems that include examining datasets for biases and documenting data preparation, and synthetic data generation is data preparation. Under GDPR, synthetic data derived from personal data is not automatically anonymous, because if individual records can be reconstructed or membership can be inferred, the data remain personal. Treat the claim "we used synthetic data so privacy is handled" as a hypothesis requiring evidence rather than a conclusion.

The decision order for scarce data. Collect more real data if you possibly can. Then check whether the problem is actually your metric or your threshold rather than your data. Then try class weighting and a simpler model. Then try transfer learning from a pre-trained model, which is covered in Weeks 9 to 11 and which usually beats augmentation when it is available. Then augment with simple, well-justified transformations. Reach for learned generative augmentation last, and document it heavily when you do.

Part III · Playing with this yourself

Everything in these notes can be run for nothing. This part covers three platforms that between them cover almost every experiment you will want to try this semester. Google Colab gives you a GPU in a browser. Hugging Face gives you models and datasets. Ollama gives you a language model running entirely on your own machine, which matters more than it sounds once you start thinking about data protection.

19 Google Colab

19.1 What it gives you

Colab is a hosted Jupyter notebook with a Python environment that already contains numpy, pandas, scikit-learn, matplotlib, PyTorch and TensorFlow. The free tier grants access to a GPU, typically a T4 with sixteen gigabytes of memory, subject to availability and a usage quota. Nothing is installed on your machine and nothing needs to be configured. For a postgraduate module where half the class is on a laptop without a discrete GPU, this removes the main practical obstacle to running real experiments.

19.2 The first five things to do in a new notebook

Open `colab.research.google.com` and create a notebook. Then go to Runtime and Change runtime type and select a GPU, because the default is CPU only and nobody notices until training is ten times slower than expected. Confirm it worked:

```
!nvidia-smi
```

Install anything missing. The exclamation mark runs a shell command, and `-q` keeps the output short.

```
!pip install -q dcor phik imbalanced-learn albumentations datasets transformers
```

Pin versions when you care about reproducibility, because Colab updates its base image without warning and a notebook that worked in October can fail in November. Record the versions you used in the notebook itself:

```
import sys, sklearn, numpy, pandas
print(sys.version.split()[0], numpy.__version__, pandas.__version__, sklearn.__version__)
```

Set every seed you can reach, in one cell, at the top:

```
import random, numpy as np, os
SEED = 42
random.seed(SEED); np.random.seed(SEED); os.environ["PYTHONHASHSEED"] = str(SEED)
try:
    import torch
    torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
    torch.use_deterministic_algorithms(True, warn_only=True)
except ImportError:
    pass
```

Finally, get your data in. For a small CSV, the simplest route is the upload widget or reading

directly from a URL. For anything you want to persist across sessions, mount Google Drive.

```
from google.colab import drive
drive.mount("/content/drive")
df = pd.read_csv("/content/drive/MyDrive/comp0484/customers.csv")
```

19.3 Things that will catch you out

The filesystem is ephemeral. When the runtime restarts, everything under `/content` that is not on mounted Drive is gone, including installed packages and downloaded model weights. Write outputs to Drive or download them before you close the tab.

Sessions disconnect. The free tier times out after a period of inactivity and has a maximum session length, so a training run that takes four hours will probably not survive. Checkpoint your model to Drive every epoch, and structure long jobs so they can resume.

GPU quota is not unlimited. Heavy use on the free tier results in a period where no GPU is offered. Develop and debug on a CPU runtime with a small subset of the data, and switch to GPU only for the run that matters.

Never paste an API token into a cell. Colab has a secrets manager, the key icon in the left sidebar. Store the token there, name it, enable notebook access, and read it in code:

```
from google.colab import userdata
hf_token = userdata.get("HF_TOKEN")
```

Save your work to GitHub through File and Save a copy in GitHub, which produces a committed notebook with outputs. That is a reasonable way to build the repository your lab deliverable asks for, though for anything serious you should also keep a plain `.py` version, because notebook diffs are unreadable in code review.

A Colab cell that reproduces Figure 1. Install `dcor`, paste the `chatterjee_xi` and `association_report` functions from Part I, and generate each of the six patterns with about twenty lines of numpy. Change the noise level and watch which coefficients degrade first. This is fifteen minutes of work and it will fix the ideas in Part I more firmly than rereading it.

20 Hugging Face

20.1 What is actually on it

The Hugging Face Hub hosts three kinds of thing. **Models** are pre-trained weights with a standard interface, spanning text, vision, audio and multimodal tasks. **Datasets** are versioned datasets with a loading API and, increasingly, dataset cards describing provenance. **Spaces** are hosted demo applications, which are useful for seeing what a model does before you commit to using it.

The relevance to this module is direct. For Part I there is not much on the Hub, since correlation is arithmetic. For Part II it is the main source of the models that perform text augmentation, and in Weeks 9 to 11 it becomes the source of the pre-trained models you will fine-tune.

20.2 Datasets

```

from datasets import load_dataset

ds = load_dataset("imdb") # downloads and caches
print(ds) # DatasetDict with train/test splits
print(ds["train"][0]["text"][:200], ds["train"][0]["label"])

small = ds["train"].shuffle(seed=42).select(range(2000)) # work small while developing
big = load_dataset("c4", "en", split="train", streaming=True) # too large to download

```

The `streaming=True` option iterates over a dataset without downloading it, which is the only practical way to touch anything above a few tens of gigabytes on Colab. Watch the cache, which lives in `~/cache/huggingface` and will silently fill a Colab disk.

20.3 Back-translation in ten lines

This is the concrete payoff for Part II. Two small translation models, running in a pipeline, give you a genuine paraphrase generator.

Listing 4: *Back-translation augmentation with Helsinki-NLP marian models*

```

from transformers import pipeline

to_fr = pipeline("translation", model="Helsinki-NLP/opus-mt-en-fr", device=0)
to_en = pipeline("translation", model="Helsinki-NLP/opus-mt-fr-en", device=0)

def back_translate(texts, batch_size=16):
    fr = [t["translation_text"] for t in to_fr(texts, batch_size=batch_size)]
    en = [t["translation_text"] for t in to_en(fr, batch_size=batch_size)]
    return en

originals = [
    "The support agent resolved my billing issue within a few minutes.",
    "I have been waiting three weeks for a refund and nobody has replied.",
]
for o, a in zip(originals, back_translate(originals)):
    print("orig:", o)
    print("aug :", a, "\n")

```

Run this on a handful of your own examples and read the output carefully. You are checking rule one from Part II: did the label survive? A complaint that comes back as a mild observation is a label-flipping augmentation and must be discarded. This inspection step is the whole point of the exercise, and it is the step people skip.

20.4 Contextual word substitution

```

from transformers import pipeline
fill = pipeline("fill-mask", model="distilroberta-base")

s = "The delivery was <mask> and the packaging was damaged."
for cand in fill(s, top_k=5):
    print(round(cand["score"], 3), cand["token_str"].strip(), "|", cand["sequence"])

```

Notice that the model will happily propose both "late" and "fine", which have opposite implications for a sentiment label. This is exactly the failure mode described in Part II, and seeing it once is worth more than reading about it three times. The fix is either to constrain the candidate set, to condition on the label, or to run a trained classifier over the augmented text and drop anything whose predicted label has changed.

20.5 Model cards, dataset cards and licences

Every model and dataset page has a card. Read it before use, and read it specifically for the things that will appear in your own datasheet: what the training data was, what the known limitations and biases are, what the intended use is, and what the licence permits. A surprising number of popular models carry non-commercial licences or restrictions on training other models, and a surprising number of popular datasets have provenance that would not survive an ethics review. Weeks 3 and 4 will return to this. For now, treat "I found it on the Hub" as the beginning of due diligence rather than the end of it.

Some models are gated and require you to accept terms and authenticate. Create a free account, generate a read token under your account settings, store it in Colab secrets, and pass it with `huggingface_hub.login(token=hf_token)`.

21 Ollama, or running a model on your own machine

21.1 Why bother when Colab exists

There are four reasons, and the first is the one that matters for this module. **Nothing leaves your machine.** If your project dataset contains personal data, sending it to a hosted API is a data transfer that needs a legal basis, a processing agreement and, quite possibly, a conversation you do not want to have. Running the model locally removes the question entirely. The second reason is cost, since there is no per-token charge. The third is that it works offline and does not disconnect after ninety minutes. The fourth is reproducibility, since you control the exact model version rather than an endpoint that is updated without notice.

The trade-off is capability. A model that fits on a laptop is smaller and weaker than a frontier hosted model, and you should expect to compensate with more careful prompting and more verification of output.

21.2 Getting started

Install from `ollama.com`. On Linux and macOS this is a single command, and Windows has an installer.

```
curl -fsSL https://ollama.com/install.sh | sh

ollama pull llama3.2:3b # small, fast, fine for paraphrasing
ollama pull qwen2.5:7b # stronger, needs more memory
ollama run llama3.2:3b # interactive chat in the terminal

ollama list # what you have downloaded
ollama ps # what is currently loaded in memory
ollama rm qwen2.5:7b # reclaim disk space
```

A rough sizing rule for the quantised weights that Ollama ships by default, usually a four-bit quantisation: a model with N billion parameters needs somewhere around $0.7N$ gigabytes of memory, plus overhead for the context window. A three billion parameter model runs comfortably on eight gigabytes of RAM. Seven or eight billion wants sixteen gigabytes. Anything above about fourteen billion wants a discrete GPU. If a model is too large it will run on the CPU rather than failing, which means it will work but produce a few tokens per second, and the symptom people report is "it is broken" when it is merely slow.

21.3 Talking to it from Python

Ollama runs a local HTTP server on port 11434. There is an official Python client, and there is also an OpenAI-compatible endpoint, so code written against the OpenAI SDK works by changing the base URL.

Listing 5: *Structured synthetic row generation, entirely on your own machine*

```
# pip install ollama
import ollama, json, pandas as pd

SCHEMA = {
    "type": "object",
    "properties": {
        "age": {"type": "integer"},
        "income": {"type": "integer"},
        "tenure": {"type": "integer"},
        "plan": {"type": "string", "enum": ["Basic", "Standard", "Premium"]},
        "satisfaction": {"type": "string", "enum": ["Low", "Medium", "High"]},
        "churn": {"type": "integer", "enum": [0, 1]},
    },
    "required": ["age", "income", "tenure", "plan", "satisfaction", "churn"],
}

def synth_row(seed_rows):
    prompt = (
        "You are generating additional plausible rows for a telecom customer table.\n"
        "Here are real examples:\n" + seed_rows + "\n"
        "Produce ONE new row with churn = 1. Ages 18-80, income in euro, tenure in months. "
        "It must be realistic and must not copy any example."
    )
    r = ollama.chat(
        model="llama3.2:3b",
        messages=[{"role": "user", "content": prompt}],
        format=SCHEMA, # constrained decoding: valid JSON guaranteed
        options={"temperature": 0.9, "seed": 0},
    )
    return json.loads(r["message"]["content"])

seed = df[df.churn == 1].sample(5, random_state=0).to_csv(index=False)
new_rows = pd.DataFrame([synth_row(seed) for _ in range(20)])
print(new_rows)
```

Two details in that snippet are worth noting. Passing a JSON schema to `format` constrains the decoding so the output is always parseable, which removes the most tedious part of using a language model programmatically. And the examples are drawn from the training split only, because the same leakage rule from Part II applies here: a generator that has seen your test rows has contaminated your evaluation.

21.4 What to use it for, and what not to

It is well suited to paraphrasing text for augmentation, to generating varied instructions or queries, to labelling or relabelling small batches of text where you then spot-check the results, and to generating small volumes of structured synthetic records for testing pipelines. It is poorly suited to producing the bulk of a tabular training set, where CTGAN or TabDDPM model the joint distribution properly and a language model is essentially guessing at plausible-looking values. It is also a poor choice for anything requiring factual accuracy without verification.

Evaluate the output before you train on it. Every technique in Part III produces data that looks convincing. Before any of it enters a training set, sample twenty rows or twenty sentences and read them. Check for duplicates and near-duplicates using embedding similarity. Compare the marginal distribution of each synthetic column against the real one. Confirm that the label is still correct. Then record what you did in the datasheet.

21.5 Combining the three

A workflow that suits this module well is to develop locally against a small sample using Ollama when language generation is involved, move to Colab with a GPU for anything that needs to train, and pull models and datasets from Hugging Face in both places. Ollama can be made to run inside Colab, but it is awkward and there is rarely a reason, because in Colab you already have a GPU and the Hugging Face transformers library gives you direct access to the same open-weight models with more control.

Task	Where to do it
Correlation experiments, resampling, pipelines	Colab on a CPU runtime, or your own machine
Image augmentation experiments, any training run	Colab with a GPU runtime
Back-translation, masked-language augmentation	Hugging Face transformers in Colab
Paraphrasing or generating records from sensitive data	Ollama locally, so no data leaves the machine
Finding a pre-trained model or a benchmark dataset	the Hugging Face Hub, and read the card

22 Exercises

These are not assessed. They are ordered roughly by effort, and each one targets a specific idea from the notes.

1. **Break Pearson deliberately.** Generate two hundred points with no relationship, then add five points at $(20, 20)$. Report Pearson, Spearman, Kendall, distance correlation and ζ . Then move the five points to $(20, -20)$ and repeat. Write one sentence explaining what each coefficient did and why.
2. **Find the minimum sample size.** For a relationship of the form $y = x^2 + \varepsilon$, determine empirically how many observations distance correlation needs before it reliably exceeds 0.3, and how many Chatterjee's ζ needs. Repeat each sample size fifty times and plot the distribution. This teaches you that new coefficients cost statistical power.
3. **Manufacture a Simpson's paradox** in your own project dataset, or demonstrate convincingly that none is present, by computing the correlation of your two most important numeric columns overall and then within each level of every categorical column you have.
4. **Reproduce Figure 4.** Build a dataset with random labels, then compute cross-validated performance both ways: SMOTE applied to everything before cross-validation, and SMOTE inside an imblearn pipeline. Vary the imbalance ratio and plot the size of the inflation. You will not forget this rule afterwards.
5. **Compare the alternatives to resampling.** On your own imbalanced dataset, evaluate four approaches with the same model and the same untouched test set: no treatment, class weighting, SMOTE inside a pipeline, and SMOTE with a partial sampling ratio. Report average precision rather than accuracy. Decide which one you would defend in a viva, and why.

6. **Audit an augmentation policy.** Take any published image augmentation recipe, apply it to twenty images from your domain, display them in a grid, and write down for each transformation whether it satisfies both rules from Part II for *your* problem. Remove the ones that do not.
7. **Generate and evaluate.** Use Ollama to generate fifty synthetic examples of your minority class. Then measure how good they are: check for near-duplicates with embedding cosine similarity, compare each column's distribution against the real data, and train a classifier to distinguish real from synthetic rows. If that classifier achieves high accuracy, your synthetic data does not resemble the real data and should not be used.

23 References and further reading

Correlation

- Anscombe, F. J. (1973). Graphs in statistical analysis. *The American Statistician*. The original four datasets.
- Matejka, J. and Fitzmaurice, G. (2017). Same stats, different graphs. The Datasaurus Dozen, a modern and more entertaining version of Anscombe.
- Székely, G., Rizzo, M. and Bakirov, N. (2007). Measuring and testing dependence by correlation of distances. *Annals of Statistics*. Distance correlation.
- Chatterjee, S. (2021). A new coefficient of correlation. *Journal of the American Statistical Association*.
- Reshef, D. et al. (2011). Detecting novel associations in large data sets. *Science*. MIC. Read alongside Simon, N. and Tibshirani, R. (2014), a short and pointed critique.
- Kraskov, A., Stögbauer, H. and Grassberger, P. (2004). Estimating mutual information. *Physical Review E*.
- Pearl, J. and Mackenzie, D. (2018). *The Book of Why*. An accessible route into causal inference; Pearl's *Causality* is the technical treatment.
- Baak, M. et al. (2020). A new correlation coefficient between categorical, ordinal and interval variables. The ϕ_K paper.

Data augmentation

- Chawla, N. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *JAIR*.
- He, H. et al. (2008). ADASYN. And Han, H. et al. (2005) for Borderline-SMOTE.
- Zhang, H. et al. (2018). mixup: Beyond empirical risk minimization. *ICLR*.
- Yun, S. et al. (2019). CutMix. And DeVries and Taylor (2017) for Cutout.
- Cubuk, E. et al. (2019, 2020). AutoAugment and RandAugment. Müller and Hutter (2021) for TrivialAugment.
- Wei, J. and Zou, K. (2019). EDA: Easy Data Augmentation techniques. *EMNLP*.
- Park, D. et al. (2019). SpecAugment. *Interspeech*.
- Xu, L. et al. (2019). Modeling tabular data using conditional GAN. CTGAN and TVAE.
- Kotelnikov, A. et al. (2023). TabDDPM: Modelling tabular data with diffusion models.

-
- Chen, T. et al. (2020). A simple framework for contrastive learning of visual representations. SimCLR, and the augmentation ablations in particular.
 - Sohn, K. et al. (2020). FixMatch. And Xie, Q. et al. (2020) for Unsupervised Data Augmentation.
 - Wang, Y. et al. (2023). Self-Instruct: Aligning language models with self-generated instructions.
 - Shumailov, I. et al. (2024). AI models collapse when trained on recursively generated data. *Nature*.
 - Gebru, T. et al. (2021). Datasheets for datasets. *Communications of the ACM*. The documentation standard referenced in the lecture.

Documentation and tools

- scikit-learn User Guide, chapters on Preprocessing, Pipelines and Cross-validation.
- imbalanced-learn documentation, particularly the section on common pitfalls, which covers the pipeline issue described in Part II.
- Hugging Face course at huggingface.co/learn, free and well made.
- Ollama documentation at github.com/ollama/ollama/tree/main/docs for the API reference.
- Albumentations documentation, which includes a visual explorer for every transformation.

Every figure in this document was produced by a single seeded script, in the same spirit as `make_assets.py` from the lecture. Change one parameter at a time and rerun it. That habit, more than any individual technique in these notes, is what separates an analysis you can defend from one you merely hope is correct.